9. Modelo de asignación de tareas

9.1. Objetivo

Formalizar cómo, dado un grafo dinámico $G(t) = (V(t), E(t))$ y una tarea Tj entrante, el sistema decide a que nodo $i^* \in V(t)$ despachar esa tarea, usando únicamente información local disponible en la red.

9.2. Formalización del problema

Sea $V(t)$ el conjunto de nodos activos en el instante t (obtenido de TablaPares). Cada nodo $i \in V(t)$ tiene asociado su vector de estado:

$Si(t) = [Ci(t), Mi(t), Bi(t), Ri(t), Ai(t)]$

Antes de evaluar la aptitud, se filtra un conjunto de candidatos aptos:

$Vcand(t) = \{ i \in V(t) \mid Ai(t) = \text{verdadero} \land Ci(t) \geq Creq(Tj) \land Mi(t) \geq Mreq(Tj) \}$

Esto hace que solo se consideran nodos disponibles que cumplen los requisitos mínimos de la tarea Tj.

9.3. Normalización de las métricas

Como Ci, Mi, Bi y Li se miden en unidades distintas, se normaliza cada métrica al rango [0,1] usando min-max sobre los candidatos actuales:

$X'i = (Xi - Xmin) / (Xmax - Xmin)$

donde $X \in \{C, M, B\}$ se normaliza de forma directa y L (latencia) se normaliza de forma inversa:

$L'i = (Lmax - Li) / (Lmax - Lmin)$

Ri (reputación) ya se puede definir de origen en el rango [0,1], por lo que no requiere normalización adicional.

9.4. Función de aptitud

Se toman los criterios con mas peso la Ci(CPU disponible) y Mi(memoria), debido a que la carga de trabajo son modelos de IA, un nodo con mucha CPU y memoria libres puede ejecutar la tarea aunque tenga algo más de latencia o menor reputación acumulada

Por tanto en el esquema de pesos Ci y Mi concentran el 80% del peso total:

Peso	Criterio	Valor
w1	CPU disponible	0.40
w2	Memoria libre	0.40
w3	Ancho de banda	0.10
w4	Reputación	0.05
w5	Penalización por latencia	0.05

Reescribiendo la función de la sección 3 con las métricas normalizadas:

$F(i, j) = w1 \cdot C'i + w2 \cdot M'i + w3 \cdot B'i + w4 \cdot Ri - w5 \cdot (1 - L'i)$

con la restricción $w1 + w2 + w3 + w4 + w5 = 1$

El nodo asignado es:

$i^* = \text{argmax}_{i \in Vcand(t)} F(i, j)$

9.5. Algoritmo de selección

Algoritmo en pseudocódigo, método “seleccionarNodo(tarea): Nodo”, de la clase “PlanificadorDistribuido”:

```
funcion seleccionarNodo(tarea Tj, V(t)):
    Vcand = filtrar(V(t), i -> i.disponibilidad == verdadero
                      AND i.cpu >= Tj.cpuRequerida
                      AND i.memoria >= Tj.memRequerida)

    si Vcand.vacio():
        retornar NULO   // se encola Tj y se reintenta en el siguiente ciclo

    normalizar(Vcand)    // min-max sobre C, M, B, L

    mejor = NULO
    mejorScore = -infinito

    para cada nodo i en Vcand:
        score = w1*i.C' + w2*i.M' + w3*i.B' + w4*i.R - w5*(1 - i.L')
        si score > mejorScore:
            mejor = i; mejorScore = score
```

```
        sino si score == mejorScore:
            mejor = desempatar(mejor, i)

    retornar mejor

funcion desempatar(a, b):
    si a.latencia != b.latencia:
        retornar el de menor latencia
    retornar el de mayor reputacion
```

9.6. Complejidad

Para una tarea Tj con |V(t)| = n vecinos conocidos, el filtrado es O(n) y la evaluación de F(i,j) sobre los candidatos es O(n), por lo tanto la selección completa es O(n) por tarea

9.7. Ejemplo numérico

Tabla con estados de nodos de ejemplo:

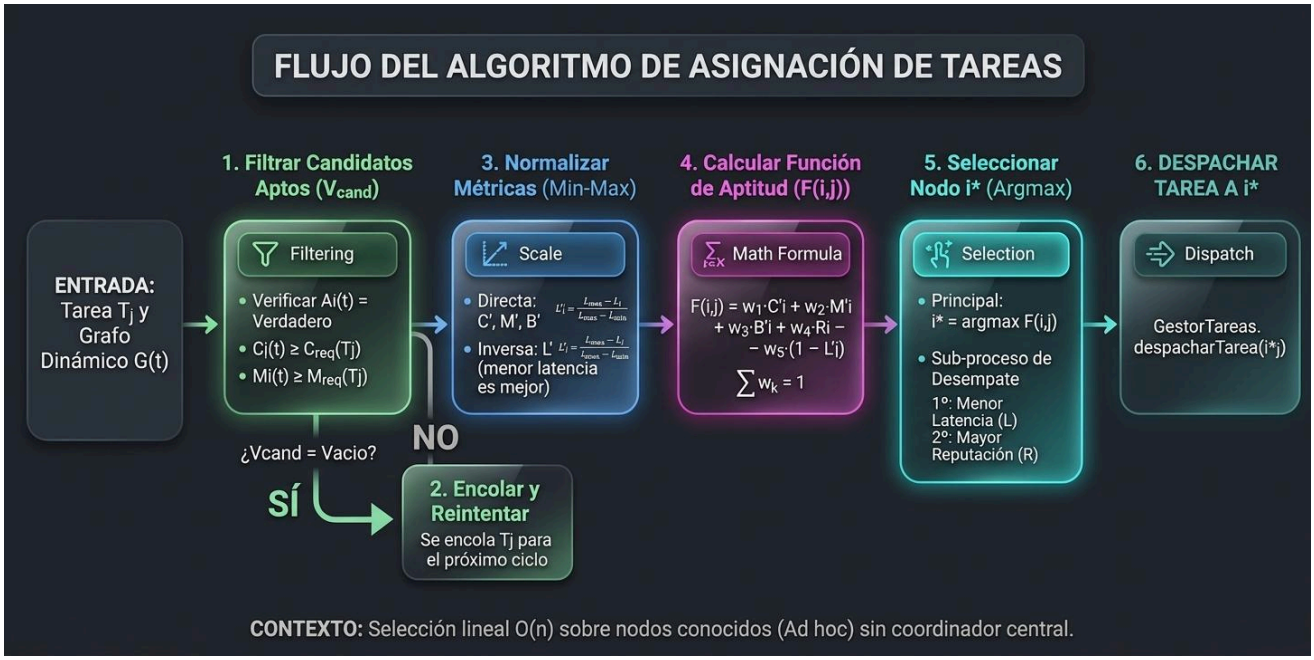
Nodo	CPU disponible	Memoria	Ancho de banda	Reputacion	Latencia(ms)
N1	80%	4 GB	50 Mbps	0.9	20
N2	60%	8 GB	100 Mbps	0.7	10
N3	90%	2 GB	30 Mbps	0.8	40

Usando las métricas normalizadas de antes(C', M', B', R, L'):

- N1 (C'=0.67, M'=0.33, B'=0.29, R=0.9, L'=0.67): F = $0.4(0.67)+0.4(0.33)+0.1(0.29)+0.05(0.9)-0.05(0.33) \approx 0.46$
- N2 (C'=0, M'=1, B'=1, R=0.7, L'=1): F = $0+0.4(1)+0.1(1)+0.05(0.7)-0 \approx 0.54$
- N3 (C'=1, M'=0, B'=0, R=0.8, L'=0): F = $0.4(1)+0+0+0.05(0.8)-0.05(1) \approx 0.39$

En este ejemplo i* = N2, la razón principal vista para la selección de este nodo es porque tiene la memoria máxima entre los tres candidatos, el N1a pesar de tener buena CPU y memoria intermedias no alcanza a N2

Representación visual del algoritmo:


FLUJO DEL ALGORITMO DE ASIGNACIÓN DE TAREAS
ENTRADA: Tarea Tj y Grafo Dinámico G(t)
1. Filtrar Candidatos Aptos (Vcand)
Filtering
• Verificar Ai(t) = Verdadero
• Ci(t) ≥ Creq(Tj)
• Mi(t) ≥ Mreq(Tj)
¿Vcand = Vacio?
SÍ
NO
2. Encolar y Reintentar
Se encola Tj para el próximo ciclo
3. Normalizar Métricas (Min-Max)
Scale
• Directa: C', M', B'
• Inversa: L'
(menor latencia es mejor)
4. Calcular Función de Aptitud (F(i,j))
Math Formula
F(i,j) = w1·C'i + w2·M'i + w3·B'i + w4·Ri − − w5·(1 − L'i)
∑wk = 1
5. Seleccionar Nodo i* (Argmax)
Selection
• Principal: i* = argmax F(i,j)
• Sub-proceso de Desempate
1º: Menor Latencia (L)
2º: Mayor Reputación (R)
6. DESPACHAR TAREA A i*
Dispatch
GestorTareas.despacharTarea(i*j)
CONTEXTO: Selección lineal O(n) sobre nodos conocidos (Ad hoc) sin coordinador central.